

PRÁCTICA N.º 4 — HOJA DE EVIDENCIAS

GraphQL y el patrón Backend for Frontend

COM-600 · Microservicios · Gestión 2/2026 · Tema 4 — Comunicación entre Microservicios

Nombre y apellidos	David Franz Soliz Ortega
Código universitario (CU)	35-5261
Carrera y paralelo	Ing. de sistemas
Fecha de entrega	10/09/2026
Sistema operativo, versión de Node y de MySQL	Windows 10 64 bits, node 24.18.1,
Repositorio (URL)	

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Entorno y proyecto base	1, 2	1	X
Laboratorio 1 — El esquema como contrato	3, 4	2	X
Laboratorio 2 — Resolvers: de dónde salen los datos	5, 6	3	X
Laboratorio 3 — Mutaciones: cambiar el estado y devolver algo útil	7, 8	4	X
Laboratorio 4 — Composición: aquí aparece el Backend for Frontend	9, 10, 11	5	X
Laboratorio 5 — El problema N+1 y la carga por lotes	12, 13	6	X
Laboratorio 6 — Errores, límites y seguridad básica	14, 15	7	X
Laboratorio 7 — Comparar con REST midiendo, no opinando	16, 17, 18, 19	8	X

PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Entorno y proyecto base

Captura N.º 1: salida de los cuatro comandos anteriores, con el JSON de usuarios visible.

```
PS C:\Users\hp> npm --version
12.0.2
PS C:\Users\hp> node --version
v24.18.1
PS C:\Users\hp> & "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" --version
C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe Ver 8.0.32 for Win64 on x86_64 (MySQL Community Server - GPL)
PS C:\Users\hp> curl http://localhost:3000/usuarios
curl : {"error":{"codigo":"RUTA_NO_ENCONTRADA","mensaje":"Ruta inexistente","detalles":{}}}
En línea: 1 Carácter: 2
+ curl http://localhost:3000/usuarios
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand
PS C:\Users\hp> curl http://localhost:3000/salud

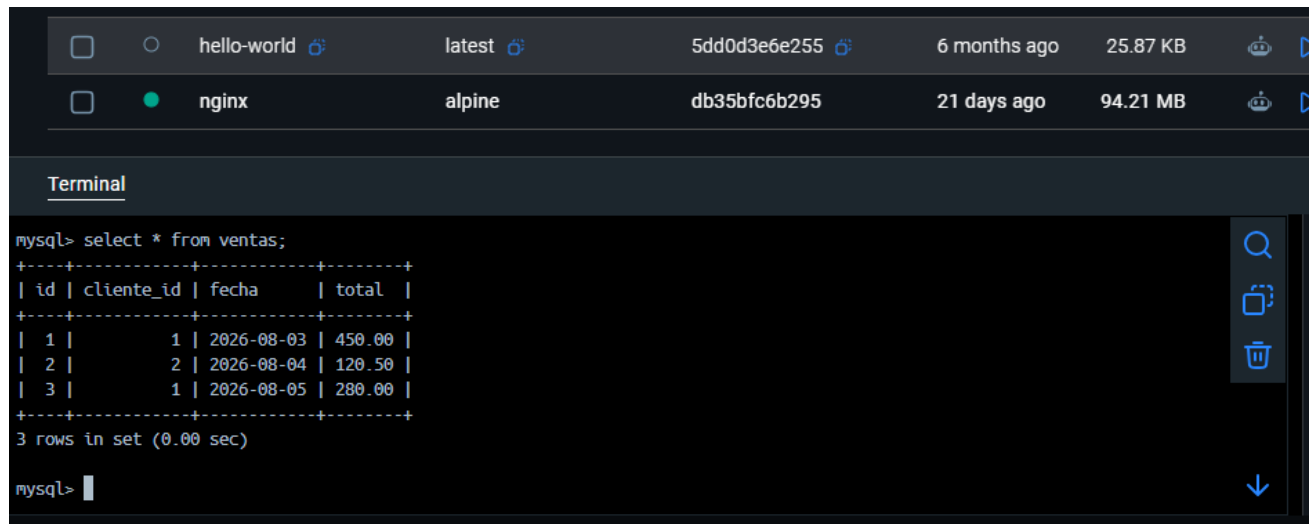
StatusCode      : 200
StatusDescription : OK
Content         : {"estado":"arriba"}
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 19
                  Content-Type: application/json; charset=utf-8
                  Date: Wed, 09 Sep 2026 18:58:12 GMT
                  ETag: W/"13-1gnfvvN/lbzfgVAAnqOr..."
Forms           : {}
Headers         : {[Connection, keep-alive], [Keep-Alive, timeout=5], [Content-Length, 19], [Content-Type, application/json; charset=utf-8]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 19

PS C:\Users\hp>
```

Pregunta N.º 1. El servicio REST de la Práctica 3 sigue funcionando exactamente igual que antes y esta práctica no le cambia una sola línea. ¿Qué gana un equipo cuando la capa nueva se pone delante en lugar de reescribir el servicio existente?

Se ahorra tiempo y se evita romper lo que ya funciona, permitiendo reutilizar la lógica anterior mientras se ofrece una interfaz más flexible para los clientes.

Captura N.º 2: consola de MySQL mostrando el resultado de `SELECT * FROM ventas;` con las tres filas insertadas.



The screenshot shows a Docker container interface. At the top, there are two containers listed: 'hello-world' and 'nginx'. Below this, a terminal window is open, showing a MySQL query and its results.

```
mysql> select * from ventas;
```

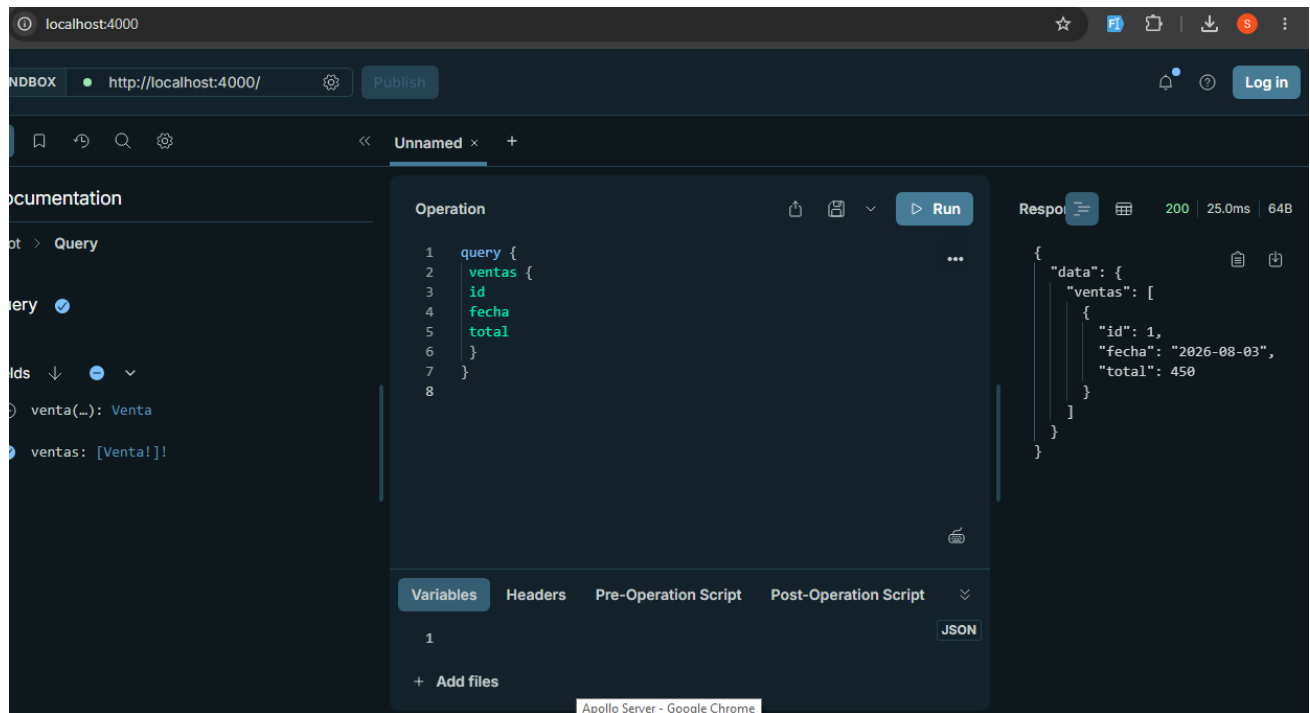
id	cliente_id	fecha	total
1	1	2026-08-03	450.00
2	2	2026-08-04	120.50
3	1	2026-08-05	280.00

```
3 rows in set (0.00 sec)
```

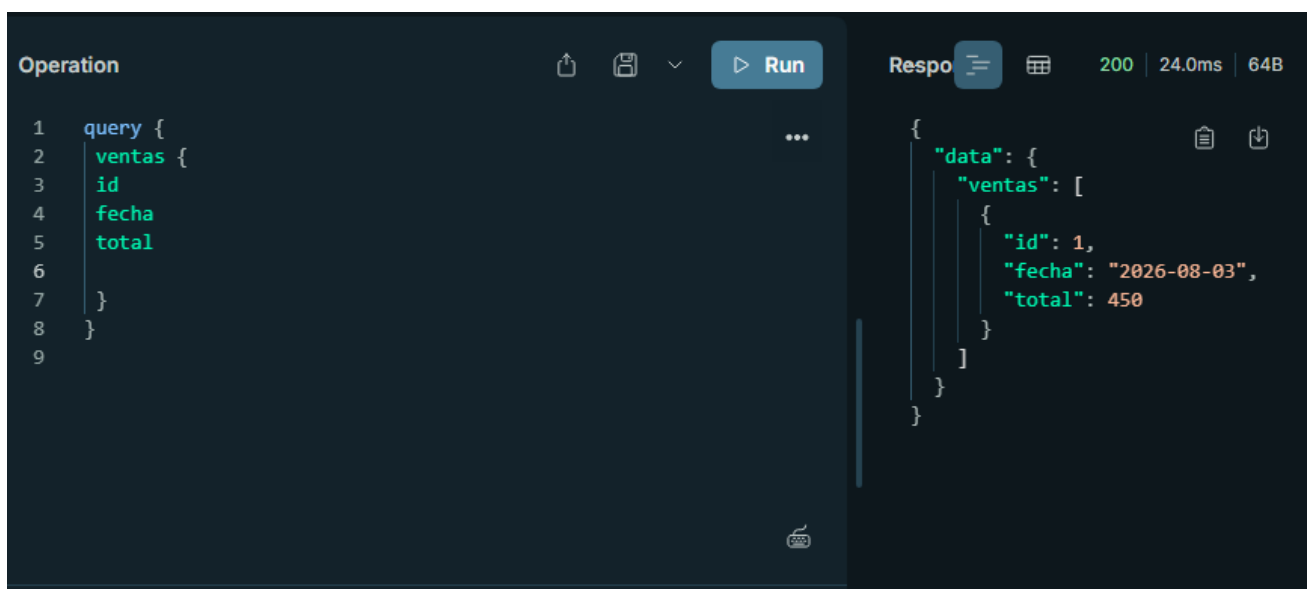
```
mysql>
```

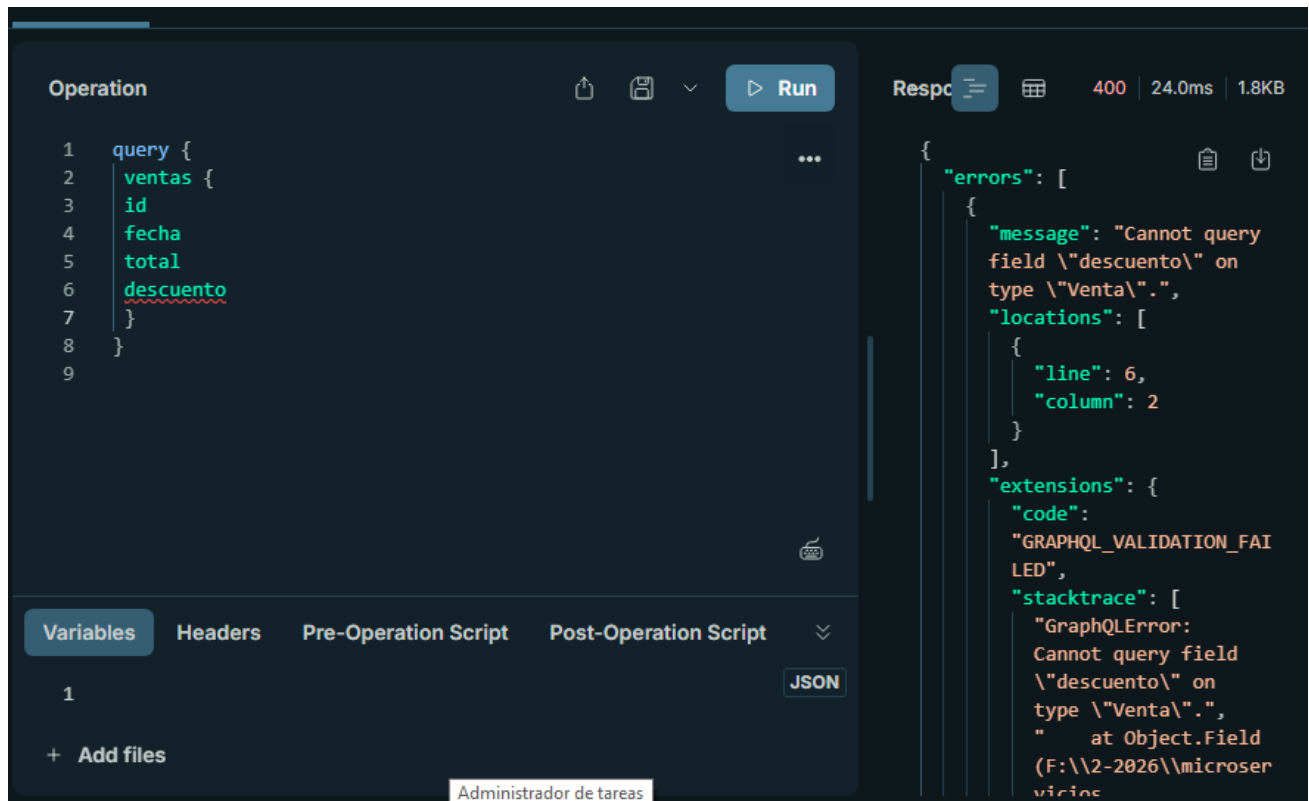
Laboratorio 1 — El esquema como contrato

Captura N.º 3: panel de documentación del explorador mostrando el tipo Venta con sus campos y sus obligaciones.



Captura N.º 4: las dos ejecuciones: la consulta válida con su respuesta y el error Cannot query field al pedir un campo inexistente.



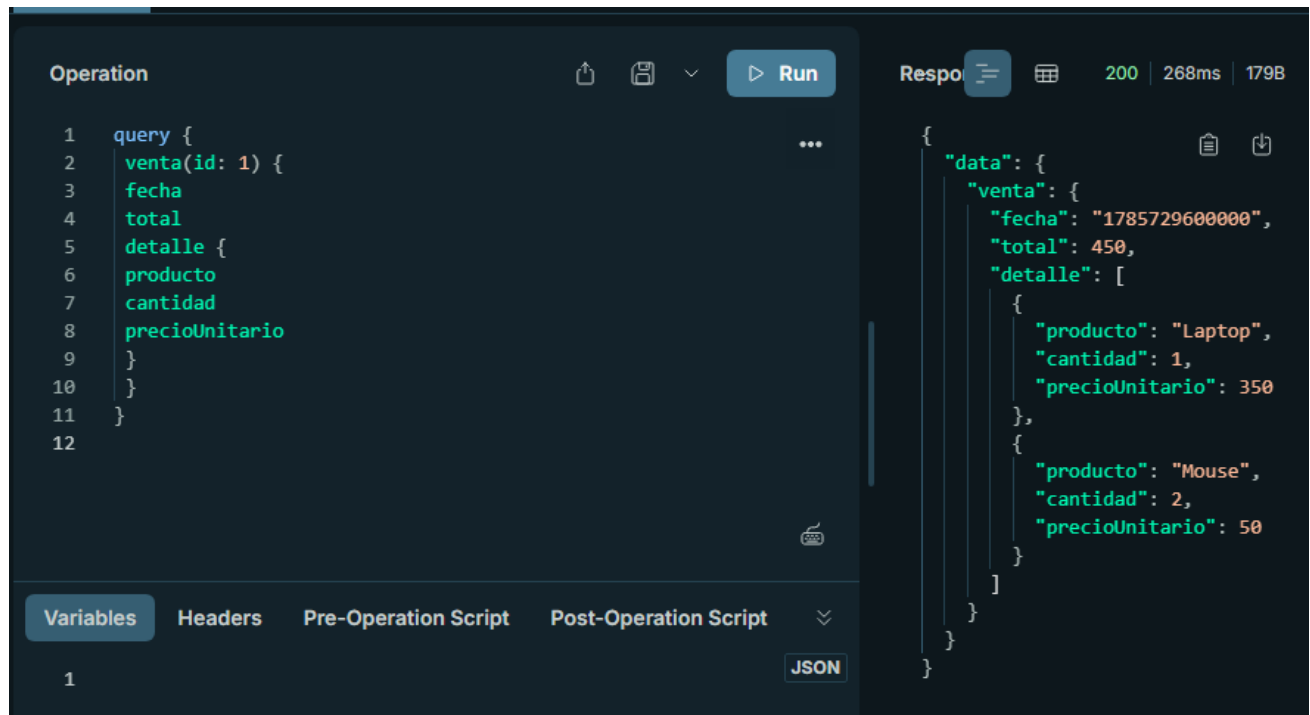


Pregunta N.º 2. En REST, pedirle a un endpoint un campo que no existe suele devolver un 200 con ese campo ausente, y el error aparece recién en el cliente. Aquí el servidor lo rechaza antes de ejecutar nada. ¿Qué implica esa diferencia para un equipo con varios consumidores del mismo servicio?

Da mayor seguridad a los equipos de desarrollo, ya que los errores se detectan de inmediato antes de que el servidor intente procesar nada, evitando fallos sorpresa en las aplicaciones cliente.

Laboratorio 2 — Resolvers: de dónde salen los datos

Captura N.º 5: la consulta con argumento y su respuesta anidada en el explorador, junto a las sentencias SQL en la terminal del servidor.



The screenshot shows a GraphQL client interface with a dark theme. On the left, the 'Operation' tab is active, displaying a query:

```
1 query {  
2   venta(id: 1) {  
3     fecha  
4     total  
5     detalle {  
6       producto  
7       cantidad  
8       precioUnitario  
9     }  
10  }  
11 }  
12
```

 Below the query, there are tabs for 'Variables', 'Headers', 'Pre-Operation Script', and 'Post-Operation Script'. The 'Variables' tab is selected, showing a single variable '1'. A 'Run' button is visible. On the right, the 'Response' tab is active, showing the JSON response:

```
{  
  "data": {  
    "venta": {  
      "fecha": "1785729600000",  
      "total": 450,  
      "detalle": [  
        {  
          "producto": "Laptop",  
          "cantidad": 1,  
          "precioUnitario": 350  
        },  
        {  
          "producto": "Mouse",  
          "cantidad": 2,  
          "precioUnitario": 50  
        }  
      ]  
    }  
  }  
}
```

 Above the response, status information is displayed: '200 | 268ms | 179B'. A 'JSON' button is located at the bottom right of the response area.

```
PS F:\2-2026\microservicios com600\practicass\practica 4\practica4-graphql> node src/servidor.js  
GraphQL escuchando en http://localhost:4000/  
[SQL 1] SELECT id, fecha, total FROM ventas WHERE id = ?  
[SQL 2] SELECT id, producto, cantidad, precio_unitario AS precioUnitario FROM detalle_venta WHERE venta_id = ?  
█
```

Captura N.º 6: la consulta con variables ejecutada dos veces: con id 2 devolviendo datos y con id 99 devolviendo nulo.

The screenshot shows the GraphQL Studio interface. On the left, the 'Operation' tab displays a query named 'ObtenerVenta' with the following code:

```
1 query ObtenerVenta($id: Int!) {  
2   venta(id: $id) {  
3     fecha  
4     total  
5   }  
6 }  
7  
8
```

Below the query, the 'Variables' tab shows the input: `{ "id": 2 }`. The 'Response' tab on the right shows the JSON output:

```
{  
  "data": {  
    "venta": {  
      "fecha": "1785816000000",  
      "total": 120.5  
    }  
  }  
}
```

At the top right of the response panel, the status is 200, 25.0ms, and 59B. The bottom of the interface includes tabs for 'Variables', 'Headers', 'Pre-Operation Script', and 'Post-Operation Script', with a '+ Add files' button.

The screenshot shows the GraphQL Studio interface. On the left, the 'Operation' tab displays the same query as in the previous capture:

```
1 query ObtenerVenta($id: Int!) {  
2   venta(id: $id) {  
3     fecha  
4     total  
5   }  
6 }  
7  
8
```

Below the query, the 'Variables' tab shows the input: `{ "id": 99 }`. The 'Response' tab on the right shows the JSON output:

```
{  
  "data": {  
    "venta": null  
  }  
}
```

At the top right of the response panel, the status is 200, 15.0ms, and 24B. The bottom of the interface includes tabs for 'Variables', 'Headers', 'Pre-Operation Script', and 'Post- Operation Script', with a '+ Add files' button.

Pregunta N.º 3. Con variables, la consulta enviada al servidor es siempre la misma cadena de texto y solo cambia el JSON de variables. Además del orden, ¿qué otro problema evita esa separación cuando el valor viene escrito por un usuario?

Evita ataques de inyección de código, ya que los datos ingresados por el usuario viajan de forma separada y nunca se mezclan directamente con la estructura de las instrucciones.

Laboratorio 3 — Mutaciones: cambiar el estado y devolver algo útil

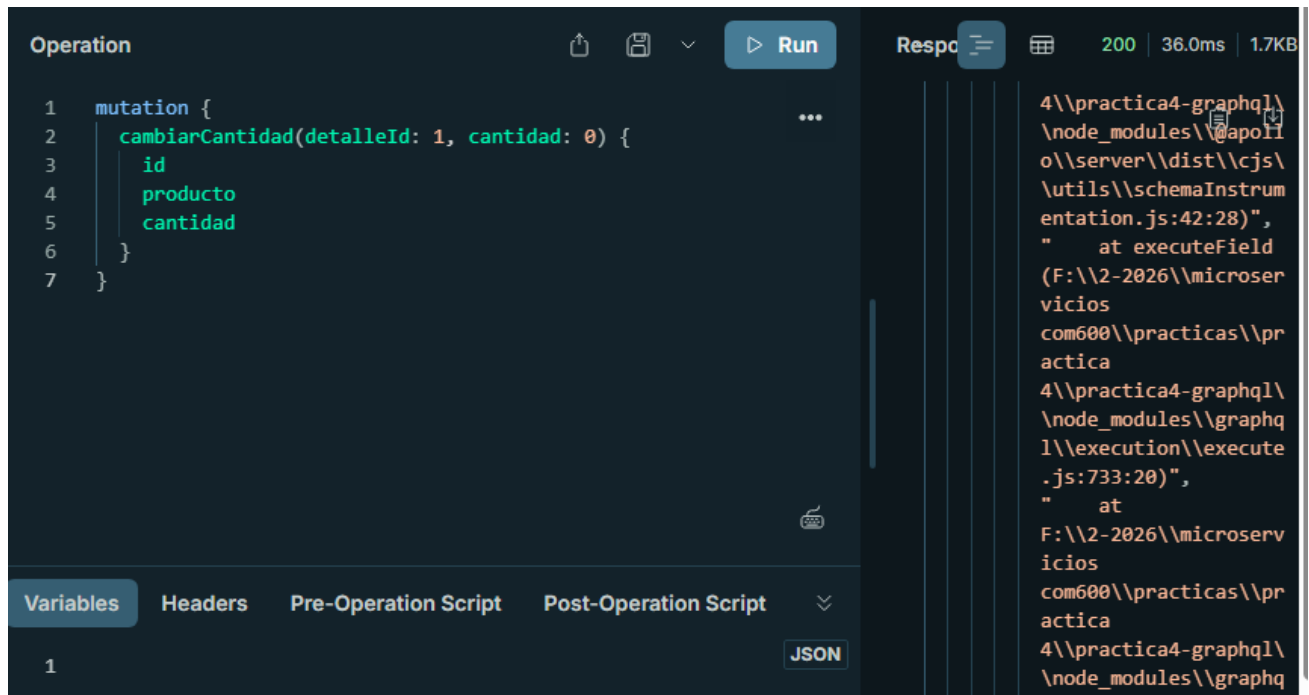
Captura N.º 7: la mutación crearVenta ejecutada, con la respuesta que trae el identificador nuevo, el total calculado y el detalle insertado.

The screenshot displays a GraphQL client interface with two main panels. The left panel, titled 'Operation', shows a GraphQL mutation query. The right panel, titled 'Response', shows the JSON response from the server. Below the operation panel, there are tabs for 'Variables', 'Headers', 'Pre-Operation Script', and 'Post-Operation Script', with 'Variables' currently selected. A 'JSON' button is visible next to the variables tab.

```
1 mutation {
2   crearVenta(input: {
3     clienteId: 2
4     fecha: "2026-08-20"
5     detalle: [
6       { producto: "Impresora", cantidad: 1, precioUnitario: 620.00 }
7       { producto: "Cartucho", cantidad: 3, precioUnitario: 85.50 }
8     ]
9   }) {
10    id
11    total
12    detalle { producto cantidad }
13  }
14 }
15
```

```
{
  "data": {
    "crearVenta": {
      "id": 5,
      "total": 876.5,
      "detalle": [
        {
          "producto": "Impresora",
          "cantidad": 1
        },
        {
          "producto": "Cartucho",
          "cantidad": 3
        }
      ]
    }
  }
}
```

Captura N.º 8: la mutación rechazada, mostrando el arreglo errors con el mensaje «La cantidad debe ser mayor que cero».



Pregunta N.º 4. Compare esta respuesta con la que daría su servicio REST de la Práctica 3 ante la misma situación. ¿Qué pierde un cliente que solo mira el código de estado HTTP cuando habla con un servidor GraphQL?

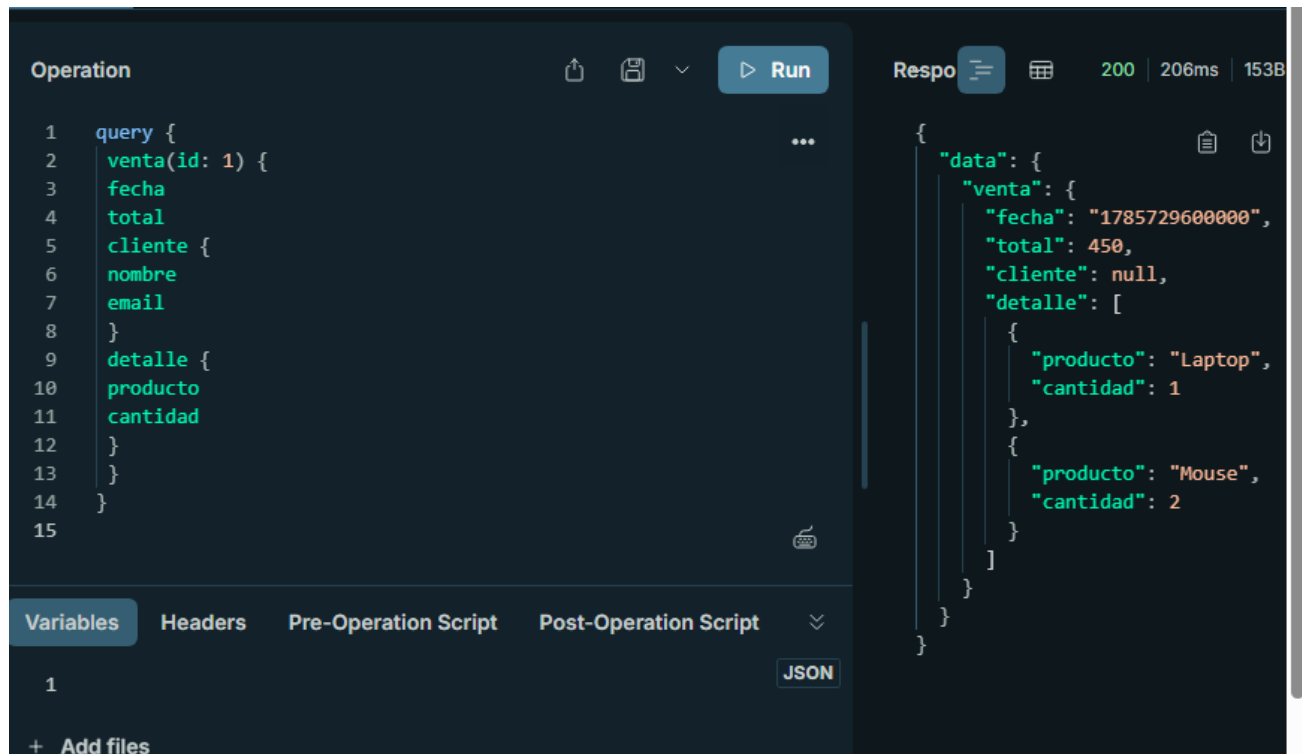
Pierde la capacidad de detectar fallos con solo mirar el estado HTTP, ya que GraphQL suele responder con un código de éxito (200) aunque existan errores internos ocultos.

Laboratorio 4 — Composición: aquí aparece el Backend for Frontend

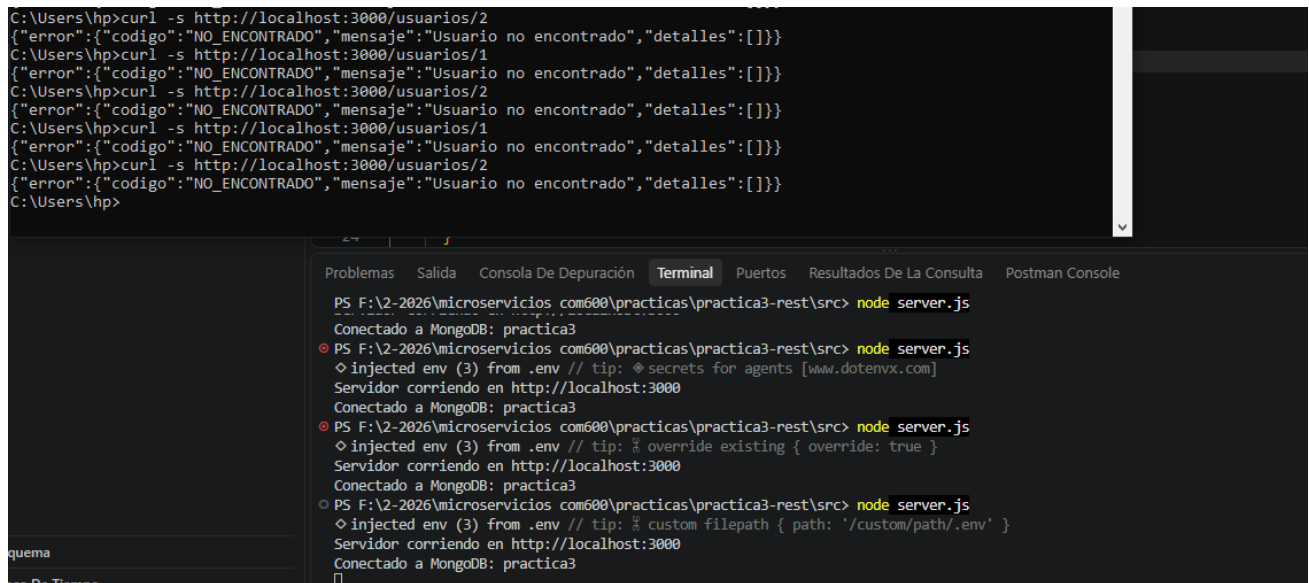
Captura N.º 9: las dos respuestas del servicio REST de la Práctica 3, con los usuarios 1 y 2.

```
C:\Users\hp>curl -s http://localhost:3000/usuarios/1
{"error":{"codigo":"NO_ENCONTRADO","mensaje":"Usuario no encontrado","detalles":[]}}
C:\Users\hp>curl -s http://localhost:3000/usuarios/2
{"error":{"codigo":"NO_ENCONTRADO","mensaje":"Usuario no encontrado","detalles":[]}}
C:\Users\hp>
```

Captura N.º 10: la consulta compuesta y su respuesta, con el nombre del cliente traído del servicio REST dentro del mismo objeto venta.



Captura N.º 11: la respuesta parcial: data con los campos que sí se resolvieron y errors indicando el fallo del campo cliente.



```
C:\Users\hp>curl -s http://localhost:3000/usuarios/2
{"error":{"codigo":"NO_ENCONTRADO","mensaje":"Usuario no encontrado","detalles":[]}}
C:\Users\hp>curl -s http://localhost:3000/usuarios/1
{"error":{"codigo":"NO_ENCONTRADO","mensaje":"Usuario no encontrado","detalles":[]}}
C:\Users\hp>curl -s http://localhost:3000/usuarios/2
{"error":{"codigo":"NO_ENCONTRADO","mensaje":"Usuario no encontrado","detalles":[]}}
C:\Users\hp>curl -s http://localhost:3000/usuarios/1
{"error":{"codigo":"NO_ENCONTRADO","mensaje":"Usuario no encontrado","detalles":[]}}
C:\Users\hp>curl -s http://localhost:3000/usuarios/2
{"error":{"codigo":"NO_ENCONTRADO","mensaje":"Usuario no encontrado","detalles":[]}}
C:\Users\hp>
```

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest\src> node server.js
Conectado a MongoDB: practica3
PS F:\2-2026\microservicios com600\practicas\practica3-rest\src> node server.js
◇ injected env (3) from .env // tip:  secrets for agents [www.dotenvx.com]
Servidor corriendo en http://localhost:3000
Conectado a MongoDB: practica3
PS F:\2-2026\microservicios com600\practicas\practica3-rest\src> node server.js
◇ injected env (3) from .env // tip:  override existing { override: true }
Servidor corriendo en http://localhost:3000
Conectado a MongoDB: practica3
PS F:\2-2026\microservicios com600\practicas\practica3-rest\src> node server.js
◇ injected env (3) from .env // tip:  custom filepath { path: '/custom/path/.env' }
Servidor corriendo en http://localhost:3000
Conectado a MongoDB: practica3
```

Pregunta N.º 5. Su capa GraphQL ahora depende de que el servicio REST esté vivo. Nombre dos consecuencias concretas de esa dependencia y una medida que usted tomaría para que la caída del servicio de usuarios no arrastre a toda la pantalla.

Si el servicio REST se vuelve lento, toda la consulta de GraphQL se arrastra y la interfaz del usuario se congela esperando esa respuesta y la sección de la pantalla que depende de esos datos se queda completamente en blanco o muestra un mensaje de error en lugar de cargar la información.
Medidas: Implementar respuestas de respaldo o un sistema de caché para que, si el servicio REST cae, la aplicación pueda mostrar información guardada previamente o datos vacíos controlados en lugar de romper toda la pantalla.

Laboratorio 5 — El problema N+1 y la carga por lotes

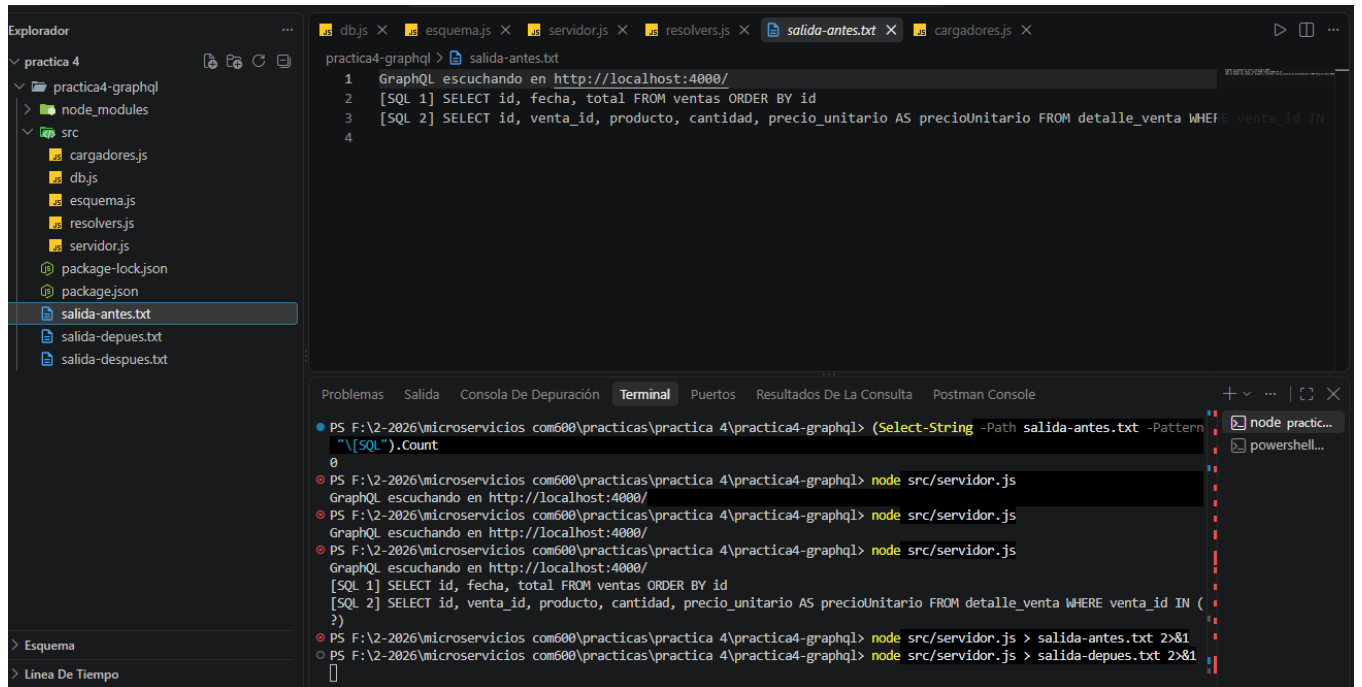
Captura N.º 12: terminal del servidor mostrando la sentencia de ventas seguida de una sentencia de detalle_venta por cada venta.



Pregunta N.º 6. La consulta responde bien y ningún registro de errores se queja. ¿Por qué este problema suele aparecer recién en producción y no en la máquina del desarrollador?

En la computadora del desarrollador hay muy pocos datos de prueba y todo va rápido, pero en producción hay miles de registros y muchos usuarios al mismo tiempo, lo que hace que los pequeños retrasos se acumulen y saturen el sistema.

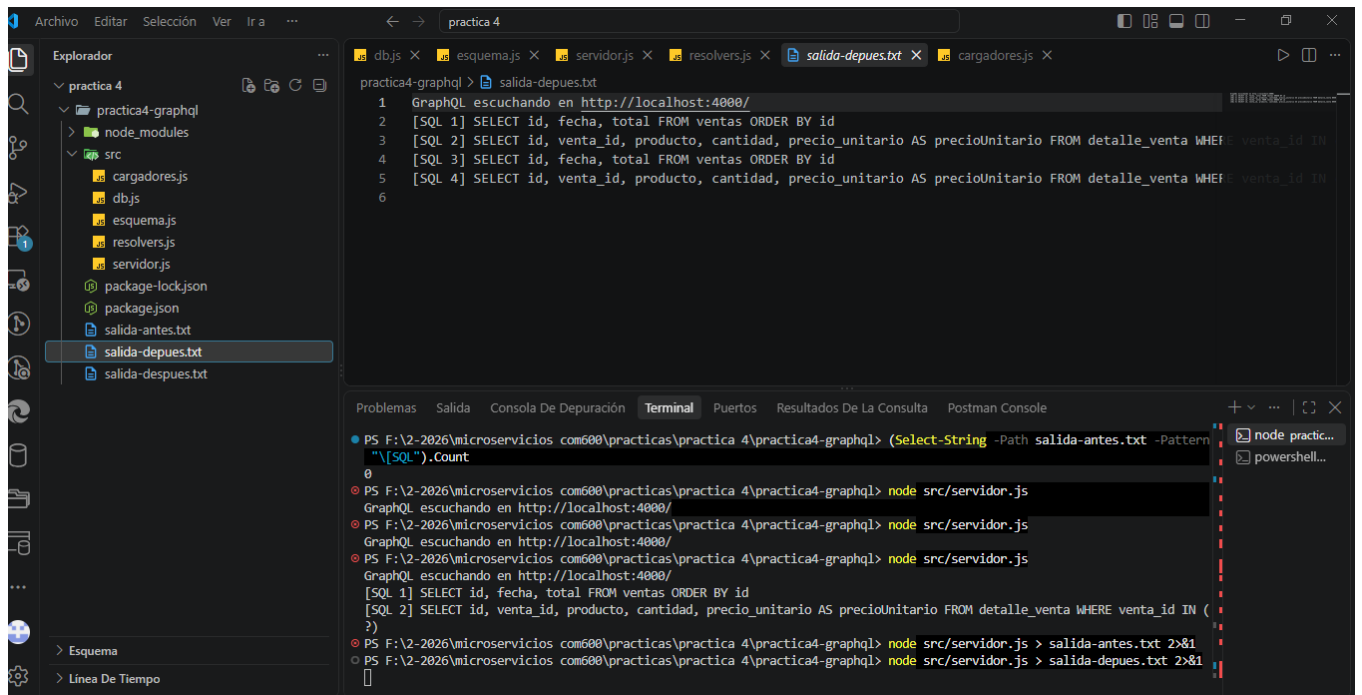
Captura N.º 13: los dos registros del servidor lado a lado: el de antes, con una sentencia por cada venta, y el de después, con una única sentencia WHERE venta_id IN (...), junto a los dos conteos.



The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure for 'practica 4'. The file 'salida-antes.txt' is selected. The main editor area shows the content of 'salida-antes.txt', which contains GraphQL query logs. The terminal at the bottom shows the execution of 'node src/servidor.js' and the output of the queries.

```
practica4-graphql > salida-antes.txt
1 GraphQL escuchando en http://localhost:4000/
2 [SQL 1] SELECT id, fecha, total FROM ventas ORDER BY id
3 [SQL 2] SELECT id, venta_id, producto, cantidad, precio_unitario AS precioUnitario FROM detalle_venta WHERE venta_id IN (
4
```

```
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> (Select-String -Path salida-antes.txt -Pattern "[SQL]").Count
0
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js
GraphQL escuchando en http://localhost:4000/
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js
GraphQL escuchando en http://localhost:4000/
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js
GraphQL escuchando en http://localhost:4000/
[SQL 1] SELECT id, fecha, total FROM ventas ORDER BY id
[SQL 2] SELECT id, venta_id, producto, cantidad, precio_unitario AS precioUnitario FROM detalle_venta WHERE venta_id IN (
?)
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js > salida-antes.txt 2>&1
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js > salida-depues.txt 2>&1
```



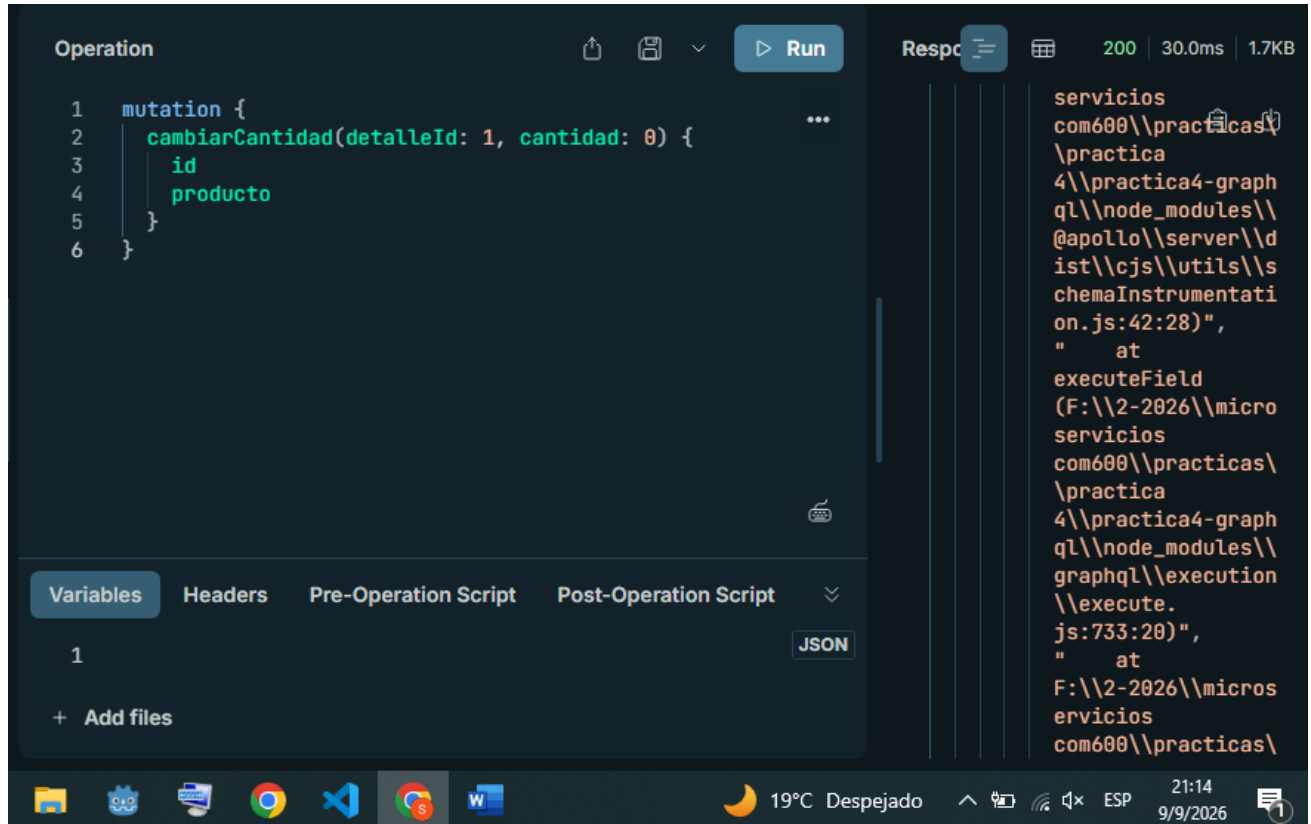
The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure for 'practica 4'. The file 'salida-depues.txt' is selected. The main editor area shows the content of 'salida-depues.txt', which contains GraphQL query logs. The terminal at the bottom shows the execution of 'node src/servidor.js' and the output of the queries.

```
practica4-graphql > salida-depues.txt
1 GraphQL escuchando en http://localhost:4000/
2 [SQL 1] SELECT id, fecha, total FROM ventas ORDER BY id
3 [SQL 2] SELECT id, venta_id, producto, cantidad, precio_unitario AS precioUnitario FROM detalle_venta WHERE venta_id IN (
4 [SQL 3] SELECT id, fecha, total FROM ventas ORDER BY id
5 [SQL 4] SELECT id, venta_id, producto, cantidad, precio_unitario AS precioUnitario FROM detalle_venta WHERE venta_id IN (
6
```

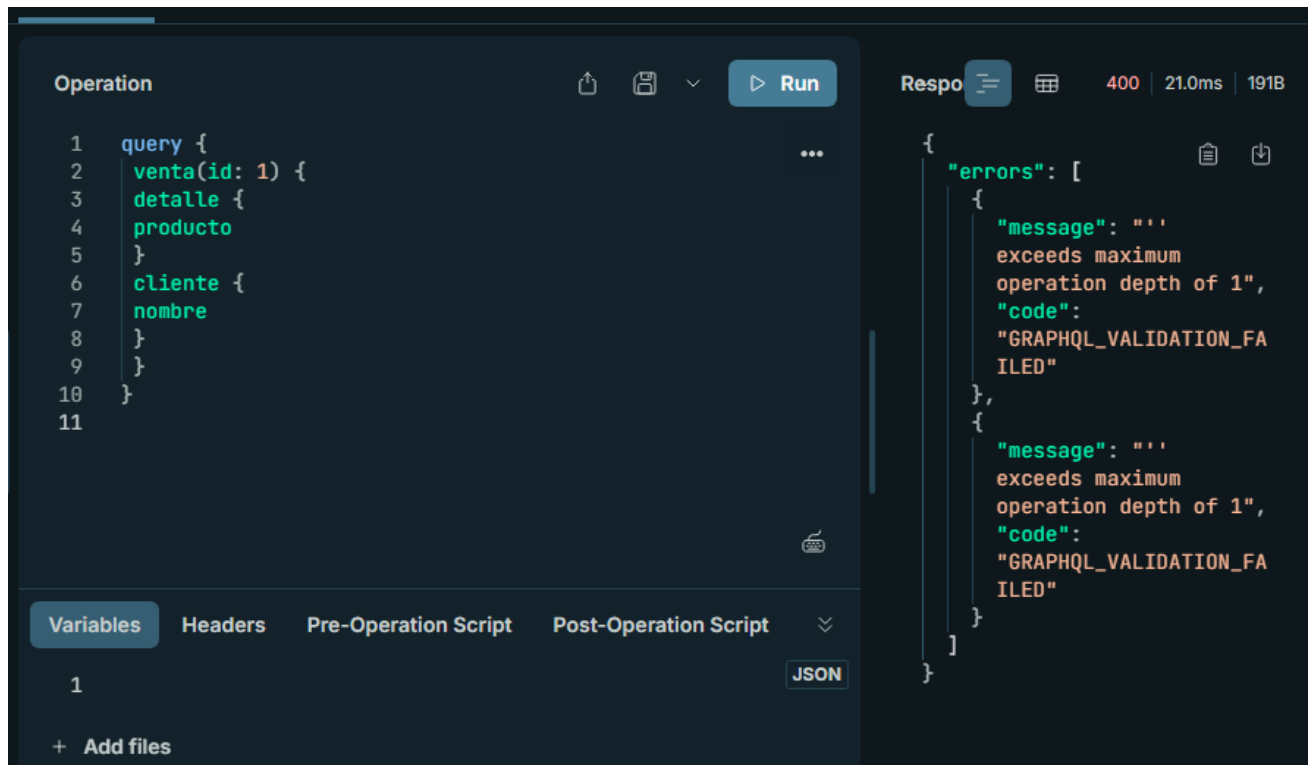
```
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> (Select-String -Path salida-antes.txt -Pattern "[SQL]").Count
0
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js
GraphQL escuchando en http://localhost:4000/
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js
GraphQL escuchando en http://localhost:4000/
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js
GraphQL escuchando en http://localhost:4000/
[SQL 1] SELECT id, fecha, total FROM ventas ORDER BY id
[SQL 2] SELECT id, venta_id, producto, cantidad, precio_unitario AS precioUnitario FROM detalle_venta WHERE venta_id IN (
?)
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js > salida-antes.txt 2>&1
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> node src/servidor.js > salida-depues.txt 2>&1
```

Laboratorio 6 — Errores, límites y seguridad básica

Captura N.º 14: la respuesta del explorador con el error resumido y, al lado, la terminal del servidor con el detalle completo.



Captura N.º 15: el error de profundidad excedida devuelto por el servidor, sin que aparezca ninguna sentencia SQL en la terminal.



Pregunta N.º 7. La profundidad no es la única medida del costo de una consulta: pedir diez mil ventas con dos niveles cuesta más que pedir una venta con seis. ¿Qué otro límite agregaría usted y sobre qué lo calcularía?

Un límite en la cantidad máxima de elementos que se pueden pedir por lista, calculado sobre el número de registros totales que una consulta puede solicitar para evitar saturar la memoria.

Laboratorio 7 — Comparar con REST midiendo, no opinando

Captura N.º 16: las tres mediciones con curl del camino REST, con sus tamaños y tiempos.

```
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> curl.exe -s -o /dev/null -w "%{size_download}
bytes %{time_total}s\n" \
>> http://localhost:3000/usuarios/1
>>
0 bytes 0.000043s
http://localhost:3000/usuarios/1 : El término 'http://localhost:3000/usuarios/1' no se reconoce como nombre de un
cmdlet, función, archivo de script o programa ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó
una ruta de acceso, compruebe que dicha ruta es correcta e inténtelo de nuevo.
En línea: 2 Carácter: 2
+ http://localhost:3000/usuarios/1
+ ~~~~~
```

Captura N.º 17: la medición con curl de la consulta GraphQL equivalente, con su tamaño y su tiempo.

```
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> curl.exe -s -o /dev/null -w "%{size_download}
bytes %{time_total}s\n" \ -H "Content-Type: application/json" \ -d '{"query":{"venta(id:1){ fecha total cliente{nombre}
}}}' \ http://localhost:4000/
ql\node_modules\body-parser\lib\types\json.js:91:21)<br> &nbsp; &nbsp; &nbsp;at F:\2-2026\microservicios com600\practicas\practi
ca 4\practica4-graphql\node_modules\body-parser\lib\read.js:162:18<br> &nbsp; &nbsp; &nbsp;at AsyncResource.runInAsyncScope (nod
e:async_hooks:227:14)<br> &nbsp; &nbsp; &nbsp;at invokeCallback (F:\2-2026\microservicios com600\practicas\practica 4\practica4-
graphql\node_modules\raw-body\index.js:238:16)<br> &nbsp; &nbsp; &nbsp;at done (F:\2-2026\microservicios com600\practicas\practi
ca 4\practica4-graphql\node_modules\raw-body\index.js:227:7)<br> &nbsp; &nbsp; &nbsp;at IncomingMessage.onEnd (F:\2-2026\microse
rvicios com600\practicas\practica 4\practica4-graphql\node_modules\raw-body\index.js:287:7)<br> &nbsp; &nbsp; &nbsp;at IncomingM
essage.emit (node:events:509:28)<br> &nbsp; &nbsp; &nbsp;at endReadableNT (node:internal\streams\readable:1729:12)<br> &nbsp; &nbsp; &nbsp;
&nbsp; &nbsp;at process.processTicksAndRejections (node:internal/process/task_queues:90:21)</pre>
</body>
</html>
1293 bytes 0.055828s
PS F:\2-2026\microservicios com600\practicas\practica 4\practica4-graphql> |
```

Captura N.º 18: la tabla de comparación completada con sus propias mediciones.

pr4 graphql > src > tablas.md

Medida	REST	GraphQL
Viajes de red del cliente	1	1
Bytes descargados en total	133 bytes	1293 bytes
Tiempo total observado	1.4936127s	0.055828s
Campos recibidos y no usados	Alto	Nulo o mínimo
Consultas SQL del lado servidor	1 sentencia fija	1 sola sentencia agrupada con IN

Pregunta N.º 8. Con sus números a la vista: ¿en qué escenario concreto de su proyecto integrador seguiría prefiriendo REST antes que GraphQL? Justifique con al menos una de las medidas que acaba de tomar.

Para la transferencia de archivos pesados o documentos dentro del sistema, ya que al tratarse de flujos simples, REST es más directo y evita el procesamiento innecesario de consultas complejas.

Captura N.º 19: la página del repositorio en GitHub mostrando la carpeta practica4-graphql con src/, db/ y el README.

[Pegue aquí la captura de pantalla]

Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

¿Qué haría distinto si tuviera que repetir esta práctica?

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

Firma del estudiante

Fecha